

How Meta Serverless Handles 11.5 Million Function Calls per Second

#94: Break Into Serverless Architecture



NEO KIM

OCT 15, 2025  PAID

Download my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines serverless architecture. You will find references at the bottom of this page if you want to go deeper.

- [Share this post](#) & I'll send you some rewards for the referrals.

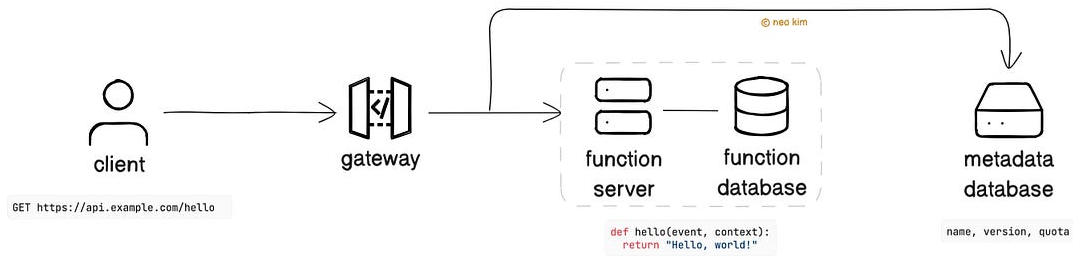
I created the block diagrams using [Eraser](#).

Once upon at Meta, engineers wrote infrastructure code whenever they launched new services.

And some services did a ton of concurrent processing during peak traffic. (Think of scheduling notifications or thumbnail creation.)

Yet a pre-allocated infrastructure doesn't scale dynamically.

So they decided to set up a function as a service (**FaaS**)¹.



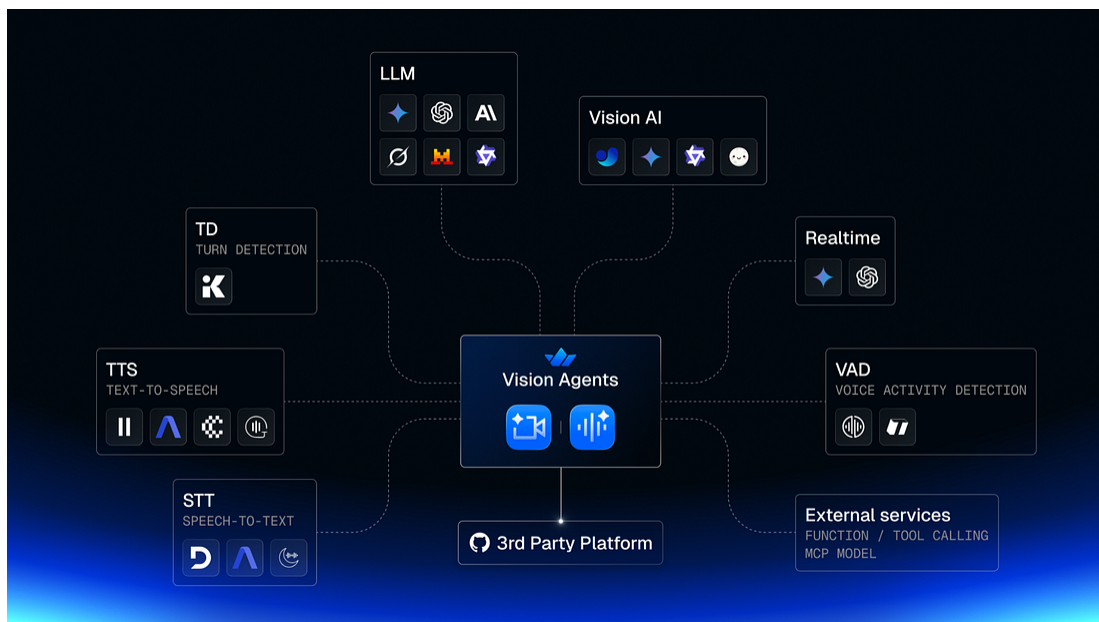
A FaaS is the serverless way to scale with increasing load.

It lets engineers focus only on writing code and handles infrastructure deployment automatically. Thus improving the engineer's productivity.

Imagine FaaS as a job queue, but managed by someone else.

Onward.

Vision Agents: Build Real-Time Video + Audio Intelligence. Open Source. Open Platform. (Sponsor).



Vision Agents is a framework for building real-time, multimodal AI that can see, hear, and respond in milliseconds.

Built for developers who want to bring true intelligence to live video without being locked into a single model or transport provider.

- Open Source: Fork it, read it, improve it.
- Open Platform: Works with Stream Video or any WebRTC-based SDK.
- Flexible Providers: Plug in OpenAI Realtime, Gemini Live, or your favorite STT/TTS and vision models.

Fully open, extensible, and ready for your next AI project.

 [Explore the Vision Agents Repo](#)

It's possible to build a hyperscale FaaS by adding many servers and putting a load balancer in front.

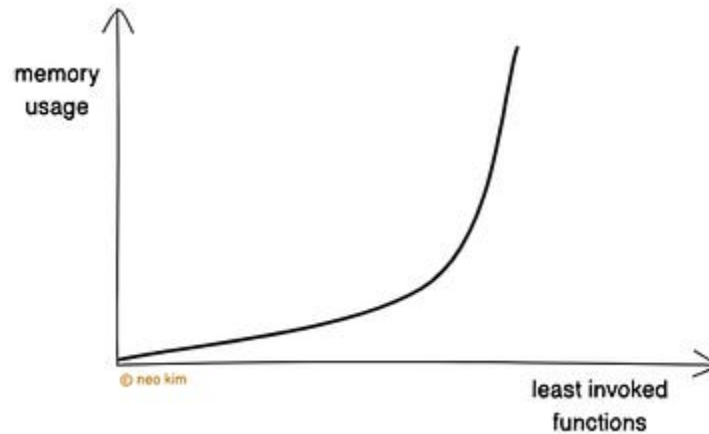
But this approach creates new problems.

Let's dive in:

1. Cost Efficiency

Restarting a server is better than keeping it idle for low costs.

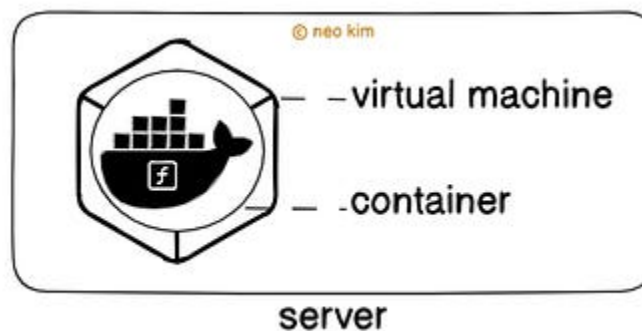
Yet a load balancer keeps servers running even if idle.



Besides:

- Idle functions use up memory² if kept warm³.
- 81% of functions get invoked at most once per minute.⁴
- And most functions run for less than a minute on average.

So this approach would cause high costs and waste computing resources. While an under-provisioned server would make a slow system.



They run functions in separate Linux processes for data isolation within the worker⁵. An idle Linux process doesn't consume many resources.

But:

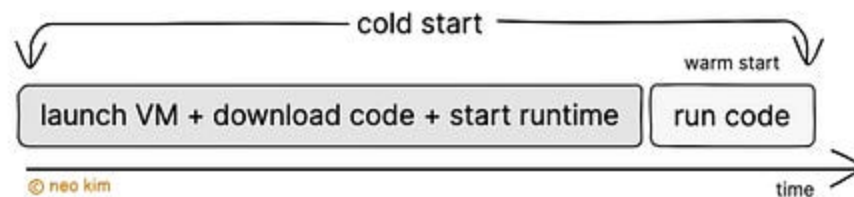
- 144 servers idle for 10 minutes → 1 server day WASTED.
- 144 servers idle for 10 minutes → 1 extra machine → same THROUGHPUT⁶.
- At Meta's scale⁷, they'd need 33,400 extra servers to achieve the same THROUGHPUT.

So it's necessary to reduce server idle time. Otherwise, they'd lose money on hardware costs.

2. Latency

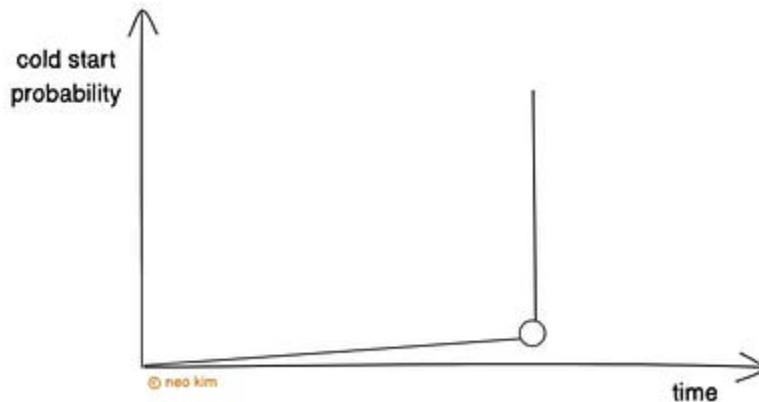
Servers crash. Also autoscaling⁸ adds new servers with varying traffic.

The creation of a new virtual machine to handle a request is called a **cold start**⁹.



Yet a cold start takes extra time¹⁰ before it can serve the first request because it's necessary to:

1. Start the virtual machine.
2. Download the container image and function code.
3. Initialize the container.
4. Initialize the language runtime and function code.
5. Do just-in-time (**JIT**) compilation¹¹.

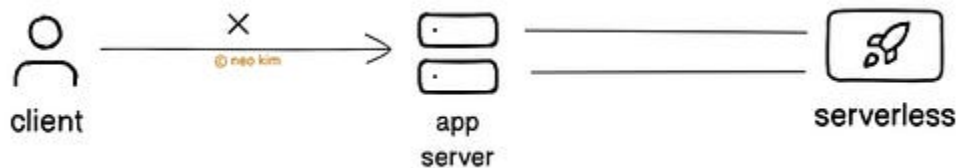


Also if the container gets shut down because a function isn't used for a period, then it has a cold start problem again. Put simply, an early shutdown of the container causes latency because of re-initialization¹².

3. Availability

Load spikes¹³ occur.

And this could overload the services that the function is calling. (Thus affecting system availability.)



Also there might be high variance¹⁴ in load. So it's necessary to:

- Scale the serverless routing layer (discovery).
- Process the functions reliably without causing retry storms¹⁵.

Besides handling errors gracefully during function execution is important.

Let's keep going!

How Serverless Works

Building a serverless platform with high CPU utilization and low latency is hard.

So smart engineers at Meta used simple techniques to solve it.

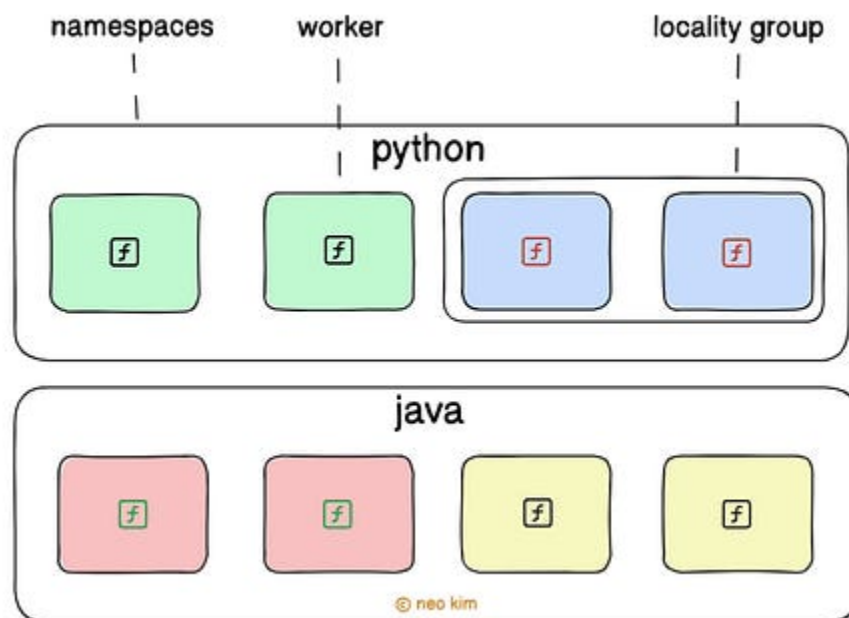
Here's how they built a hyperscale distributed operating system (XFaaS):

1. Cold Start

They run functions on a server called the **worker**.

And categorizes those workers into namespaces based on programming language. This ensures that each worker gets only compatible code and executes faster.

Also this technique gives extra security through physical isolation.



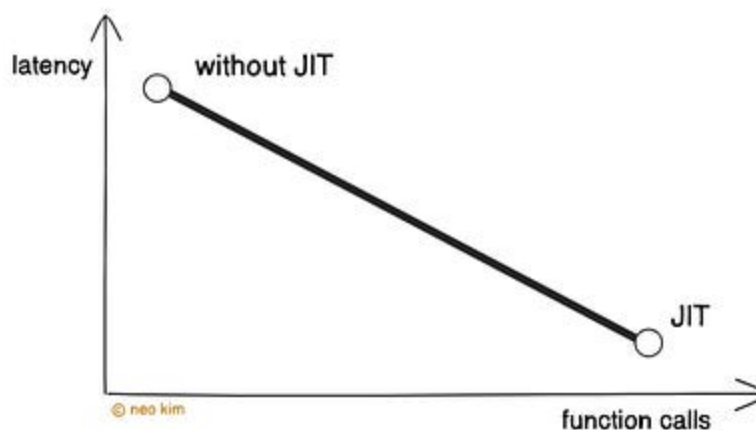
While a worker within a namespace can run any function written in that language to reduce cold start. This is called the **universal worker model**.

Yet each server doesn't run every function, but only a subset of them. This approach keeps the JIT-compiled code cache small and efficient.

Also they form **locality groups** within a namespace by grouping workers that run the *same* functions often. This approach reduces memory usage and improves caching through better JIT code reuse.

Repeated downloads increase latency and waste network bandwidth.

So they **pre-push** the function code and container images onto the server's SSDs. This avoids repeated downloads and keeps an always-active runtime.



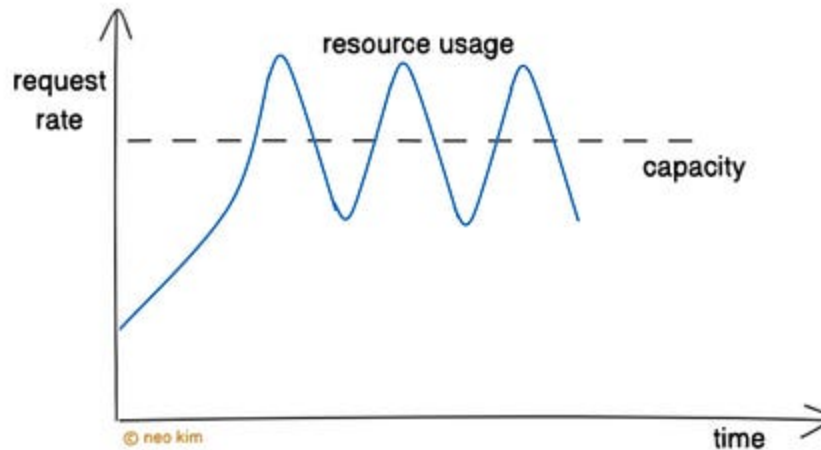
Besides they do **cooperative JIT compilation** for faster execution of functions. It means only specific workers compile new function code. Then those workers send the compiled code to ALL workers. Thus preventing redundant compilation overhead.

Ready for the next technique?

2. Backend Service Protection

There's a risk of overloading the backend services that the function is calling during a load spike.

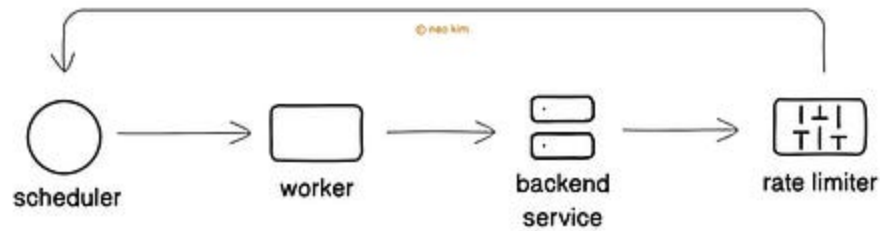
So they protect those services using **backpressure**-based throttling.



Here's how it works:

- They use a technique similar to TCP congestion control for throttling.
- Rate limiter throttles requests upon a backpressure signal from the backend service. For example, they track error, latency, and capacity.
- And then slowly increases the requests to those backend services.
- On congestion detection, it throttles requests again.

Yet it throttles only the functions that call backend services for performance.



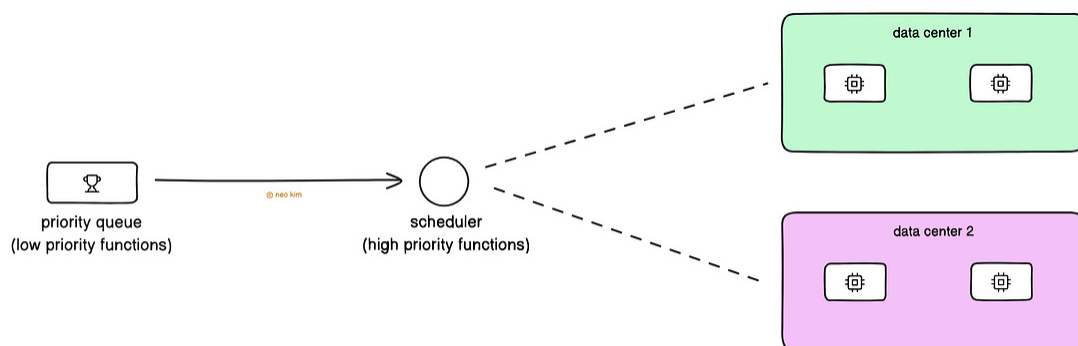
Besides it's necessary to control the number of concurrent functions calling a backend service to prevent overload.

So they start with fewer concurrent function requests. This gives backend services enough time to warm up their cache and handle the load.

3. Peak Traffic

It's necessary to prevent a function from using up the resources during peak traffic.

So they assign a limit to how many times a function can run or how many resources it can use. After a function reaches that limit, it's throttled.



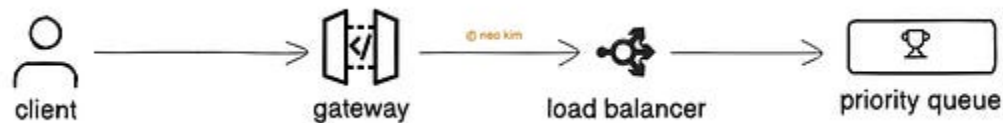
Also they distribute function execution across data centers to distribute the load evenly. And delay the low-priority functions until low-traffic hours¹⁶. Put simply, they run specific functions only when there's enough free server capacity.

Besides most services are stateless and replicated for scalability and fault tolerance. While stateful services are partitioned and replicated.

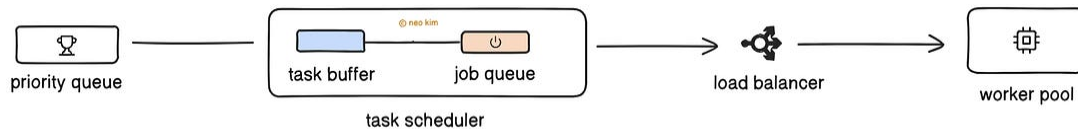
Ready for the best part?

Serverless Architecture

Here's a high-level overview:



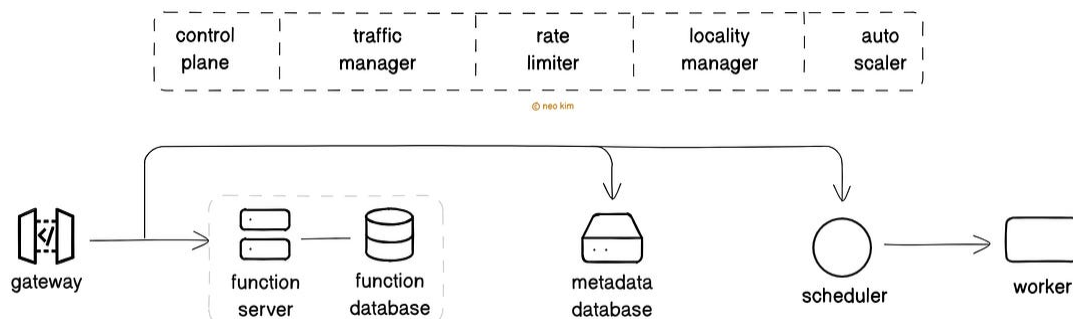
1. Client submits a function call along with its arguments to the gateway.
2. Gateway assigns a UUID¹⁷ to each function call: a long, random, and unique ID.
3. Then it stores function calls with large arguments (payload) in a distributed key-value store. And passes only their reference key for performance.
4. Also gateway checks the throttling limit and forwards requests in batches to the load balancer for efficiency.
5. Load balancer distributes function calls evenly across different data centers using their UUID. It's stored in priority queues there.
6. Priority queue arranges the functions based on their importance and execution time. Then stores pending function calls in a highly available and sharded database for fault tolerance.



7. While task scheduler periodically pulls functions from the priority queue based on current traffic and workload. It consists of the task buffer¹⁸ and the job queue (both in-memory data structures).
8. Task buffer determines function execution order based on their importance, quota¹⁹, and deadline.
9. And forwards functions to the job queue, which then distributes them across the correct pool of workers using a load balancer²⁰.
10. Task scheduler informs the priority queue after a function runs successfully, so it can remove the function from its list.

Task scheduler stops scheduling functions to a worker if it's overloaded.

Also if a function reaches its quota during busy hours, they put extra requests in the priority queue until capacity frees up. The higher the worker usage, the higher the throttling!



Besides they run some central services to coordinate the system:

- **Control plane:** manages system configuration, function metadata, and runtime parameters of services.
- **Traffic manager:** routes function requests to the correct region.

- **Rate limiter:** enforces rate limits and quotas to prevent server overload and ensure fairness during peak traffic.
- **Locality manager:** groups functions and workers to improve cache reuse and performance.
- **Autoscaler:** monitors CPU and memory usage across workers to adjust scheduling or scaling.

Yet these services are outside the critical path to prevent performance issues.

TL;DR

- **Cold Start:**
 - Keep runtimes warm for instant execution.
 - Universal workers grouped by language namespaces.
 - Pre-push function code and container images to local SSDs.
 - Locality groups improve JIT cache reuse and memory usage.
 - Cooperative JIT compilation to share compiled code across workers.
 - **Backend Service Protection:**
 - Backpressure-based throttling.
 - Limit concurrent function calls to prevent backend overload.
 - **Peak Traffic:**
 - Cross-region load balancing.
 - Delay low-priority jobs until low-traffic hours.
 - Apply quotas per function to avoid resource hogging.
-

Serverless architecture is excellent for:

- queue-based workloads.
- Time-bound functions with no persistent state across invocations.

Yet it could cost 2x as much as a server-based architecture on predictable and steady traffic. Also it isn't built for synchronous workloads. Besides there might be limitations like request-response size limits. So use it only if necessary.

Meta reduced their idle time to 1 minute.

And have a 66% average CPU utilization compared to the 27% from public FaaS of AWS and GCP. Yet they use this only to run their internal services²¹.

Remember, a small productivity gain per engineer becomes massive when many engineers and jobs are involved.



👋 Say hi on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

References

- [XFaaS: Hyperscale and Low Cost Serverless Functions at Meta](#)
- [SOSP '23 | XFaaS: Hyperscale and Low Cost Serverless Functions at Meta, Slides](#)

- [How Meta built the infrastructure for Threads](#)
 - [Holistic Configuration Management at Facebook](#)
 - [Peeking Behind the Curtains of Serverless Platforms](#)
 - [Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider, Whitepaper](#)
 - [XFaaS: Cross-platform Orchestration of FaaS Workflows on Hybrid Clouds](#)
 - [Fn Project. Open Source. Container-native. Serverless platform.](#)
-

- 1 Serverless means you don't manage servers at all (it includes databases, APIs, and so on). While FaaS means running only small pieces of code (functions) on demand within that serverless setup. I'll be using the term Serverless instead of FaaS in this post.
- 2 Virtual machines store the container, runtime, function code, and libraries in memory. So there's a lot of memory overhead at their scale, not CPU.
- 3 Keeping a function warm means leaving it running in memory so it can handle the next request quickly, without waiting to start up again.
- 4 These numbers are from the Azure case study: Serverless in the Wild - Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider.
- 5 A worker is a server that runs the function code.
- 6 Throughput means how many compute minutes are available every 15 minutes.
- 7 Meta serverless has 100,000+ servers, 10+ regions, 18,000+ different functions, and handles 1 trillion function calls a day. They support languages such as Python, Golang, Java, and C++.
- 8 Autoscaling means the system automatically adds or removes servers based on the traffic.

- 9 The server needs to perform some tasks before getting ready for the first request. Thus affecting latency and performance.
- 10 AWS Lambda keeps functions warm for 10 minutes, which means a cold start after 10 minutes. While Azure keeps functions warm for 20 minutes.
- 11 When a function runs for the first time, the system converts its code into fast, optimized machine code "just in time" to ensure efficient execution.
- 12 Late shutdown of a container wastes resources. While an early shutdown means latency because of container re-initialization.
- 13 Load spike means a sudden, short-term increase in traffic that exceeds normal levels (for example, sending notifications for a celebrity post).
- 14 Their peak traffic = 4x normal traffic. Meta's private cloud has the advantage of predictable traffic patterns.
- 15 It means running the functions safely without causing many repeated retries when something fails.
- 16 This technique is also known as time-shifting computing.
- 17 Universally Unique Identifier (**UUID**) is a special ID number that's almost guaranteed to be unique for every request.
- 18 There are many task buffers, and they're organized by priority or locality.
- 19 A quota is defined as the number of CPU cycles consumed by a function. It's assigned by the client.
- 20 The load balancer chooses the pool of workers based on memory usage, the hit rate of JIT-compiled code.
- 21 They use XFaaS in their recommendation system. For example, to show friend suggestions when someone accepts a friend request.